

Training Data Attribution During Training: Optimizer Unrolling vs. Gradient Trajectory Methods

Introduction

Training data attribution (TDA) methods seek to quantify how individual training examples (or batches) influence a model's predictions. In essence, they ask counterfactual questions like: “How would the model's behavior change if a particular training point were removed or altered?”¹. Classical approaches to TDA, such as *influence functions*, estimate this influence by analyzing a fully trained model under strong assumptions (e.g. convex loss, convergence to optimum)²³. However, modern deep learning violates many of these assumptions – models are highly non-convex, often not fully converged, and sensitive to the order of training data⁴. This has spurred new families of *training-phase* attribution techniques that operate during training or across training steps, rather than solely on a finalized model. Two prominent such families are: **(1) Optimizer-unrolling methods**, which approximate a training point's influence by differentiating through the optimization process itself, and **(2) Gradient trajectory-based methods**, like **TracIn**, which track the effect of examples over the course of training via intermediate gradient information. In this report, we explain the foundations of each approach, highlight key contributions, and discuss their computational trade-offs. We also compare these methods to classical (and Bayesian) influence functions, and outline their applicability to different model types. Finally, we illustrate common use cases – from data quality assessment and debugging to model interpretability – and how these techniques can be integrated into real-world ML workflows.

Background: Classical and Bayesian Influence Functions

Classical influence functions (Cook's influence⁵, extended by Koh & Liang 2017 to deep learning) provide a baseline for understanding data influence. The classical approach treats the model's *final* parameters as an implicit function of the training data and uses a first-order Taylor expansion to estimate the effect of removing a data point³. Intuitively, one computes how a small down-weighting of a training sample z would shift the optimal parameters $\hat{\theta}$, then estimates how that parameter shift would change the loss on a test point z_{test} . The result is a closed-form influence score:

$$\mathcal{I}(z, z_{\text{test}}) \approx -\nabla_{\theta} L(z_{\text{test}}, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}),$$

where $H_{\hat{\theta}}$ is the Hessian of the total training loss at $\hat{\theta}$ ⁶⁷. This score approximates the change in z_{test} 's loss if z were removed from training³. Classical influence functions have appealing efficiency – one can often compute $H^{-1} \nabla L(z, \hat{\theta})$ via Hessian-vector products without full retraining⁸. They were successfully applied to identify mislabeled training points and explain model predictions in early deep learning settings⁸⁹. However, they rely on assumptions that break down in realistic scenarios. They assume the model is trained to a strict optimum

and that $H_{\hat{\theta}}$ is positive-definite (invertible) ¹⁰ ¹¹. In deep networks, loss landscapes are non-convex and often *underspecified* – many parameter configurations can yield similar performance ¹². Moreover, modern training often stops early for regularization, violating optimality assumptions ¹³. As a result, classical influence scores can become unstable or inaccurate for neural networks (e.g. failing to reliably estimate influence under SGD training of CNNs ¹⁴ ¹⁵). They also ignore *training order* – treating the dataset as an unordered set – which is problematic for one-epoch or curriculum training where order matters ¹⁶ ¹⁷. In fact, classical influence functions would assign identical scores to duplicate data points regardless of when they appear in training, which does not reflect reality in non-convex, order-sensitive training regimes ¹⁸.

Bayesian influence perspectives have emerged to address some of these issues. Instead of analyzing a single deterministic model, a Bayesian view treats the model parameters as a posterior distribution given the training data ¹⁹. In this view, the influence of a data point can be considered a random variable (since it may differ across model initializations or posterior samples) ²⁰. Nguyen et al. (NeurIPS 2023) argue that in deep learning, the apparent influence of an individual sample is often swamped by variability due to random initialization and stochastic training noise ²¹. Thus, they caution that *consistent* influence across many training runs is needed to trust that a particular training example truly drives a prediction ²¹ ²². This aligns with the intuition that one should assess influence robustness under different seeds or model draws – a single run’s influence score might be a fluke. On the methodological side, *Bayesian Influence Functions (BIF)* have been proposed to avoid computing a single huge Hessian inverse. Kreer et al. (2025) introduce a “local BIF” that estimates influence using *loss landscape statistics* gathered via stochastic gradient MCMC sampling instead of explicit Hessian inversion ²³. This approach captures higher-order curvature effects and scaled to neural networks with billions of parameters ²⁴. In summary, Bayesian approaches aim to incorporate uncertainty and overcome Hessian-related issues, but they often require expensive sampling or multiple retrainings. These challenges motivate methods that operate *during training*, which we now discuss.

Optimizer-Unrolling Attribution Techniques

Optimizer-unrolling methods estimate training data influence by **differentiating through the training process itself**. Instead of only analyzing the endpoint of training, these techniques *unroll* the optimization (e.g. unroll the SGD updates) and compute how a small change in a training example or its weight would propagate through each training step to affect the final model. Conceptually, one treats the entire training run as a differentiable computation graph: the model parameters at step T (θ_T) are a function of earlier parameters and the sequence of mini-batch updates. By applying the chain rule through this sequence, one can obtain the gradient of some final objective (say, the final loss on a validation or test point) with respect to a particular training sample’s presence or value. This “training-level” differentiation yields an influence estimate that inherently accounts for the *optimizer’s trajectory* and inductive biases (e.g. early stopping, learning rate schedule, etc.) ¹² ²⁵. In theory, if one could fully unroll a training procedure with respect to each training point, it would exactly answer the leave-one-out question even for non-convex models: it simulates the counterfactual of removing that point by seeing how its absence changes the learning trajectory.

Key methods and contributions: Early work by Hara et al. (2019) introduced the notion of *SGD-influence*, redefining influence for models trained with SGD rather than assuming a global optimum ²⁶ ²⁷. They posed the question: “What if an instance were never seen by SGD – how much would the resulting model’s parameters differ?” ²⁸. Their estimator analytically traced the effect of omitting a sample through each

SGD update using a first-order Taylor expansion of the loss change at that step ²⁹ ³⁰. This method did not require the model to be at a final optimum and was thus better suited to early-stopped or non-convex scenarios ²⁷ ¹³. Subsequent research refined these ideas. For example, *SGD-Influence Estimation* methods initially summed per-epoch effects, but Shi et al. (2025) found that naively accumulating one-epoch leave-out effects over multiple epochs incurs bias ³¹ ³². They proposed an **Accumulative SGD Influence Estimator (ACC-SGD-IE)** that inserts corrective Hessian-vector terms at each re-encounter of the target sample, achieving a more accurate recursive influence estimate over long training trajectories ³³ ³⁴. In essence, each time the omitted sample *would* have appeared, they adjust for the compounded effect of its absence on subsequent updates, greatly improving accuracy in multi-epoch training ³⁵ ³³.

Another notable contribution is by Bae et al. (2024), who introduced a method called **Source** (Segmented statiOnary Unrolling for Counterfactual Estimation) ³⁶ ³⁷. Source bridges the gap between implicit differentiation (influence function-style) and explicit unrolling. The authors observed that fully unrolling training is often infeasible, yet implicit methods fail when models are under-trained or when training involves multiple stages (like pre-training + fine-tuning) ¹² ²⁵. Source therefore uses an *influence-function-like formula* to approximate an unrolled influence **efficiently** ³⁸ ³⁹. It segments the training process and applies a form of truncated unrolling in each segment, combining results in a way inspired by classical influence computations. This allows it to handle scenarios where classical influence is unreliable (e.g. non-converged models, multi-stage pipelines) while remaining much cheaper than a full unroll ¹² ⁴⁰. Empirically, Source outperformed prior TDA methods in predicting true leave-one-out effects, especially in settings with early stopping or complex training curricula where implicit methods struggle ¹² ⁴¹.

Across these methods, a common theme is **trading off exactness for tractability**. Fully differentiating through every SGD step for every training example would be exact but prohibitively expensive for deep networks. Instead, unrolling methods use approximations: *first-order* approximations (ignoring higher-order terms except via simple Hessian-vector products) ⁴², *truncated unrolling* (differentiating through only the last k steps or last epoch) ⁴³, or *segmented unrolling* as in Source. Another recent example is **Diff-In** (Tan et al., 2024), which formulates the total influence of a sample as the *cumulative sum of its influence differences between successive training iterations* ⁴⁴. By carefully approximating these per-step differences with second-order accuracy (using Hessian-gradient products), Diff-In eliminates the convexity requirement of classical influence yet keeps computation manageable ⁴⁵. Notably, Diff-In achieves *second-order precision* with a cost comparable to first-order methods, by exploiting efficient finite-difference approximations to Hessian-vector products ⁴⁶ ⁴⁷. This allowed Diff-In to scale to millions of training points and beat other baselines on tasks like data cleaning and coreset selection ⁴⁸ ⁴⁹.

Computational requirements: Optimizer-unrolling techniques are generally more computationally intensive than post-hoc influence methods. Differentiating through T training steps naively requires storing intermediate states or recomputing them, which is memory and time heavy. For example, iterative differentiation through SGD requires storing the model gradient at each step (or recomputing it in reverse), akin to backpropagation through time for potentially thousands of iterations ⁵⁰ ⁵¹. This quickly becomes infeasible for long trainings or large models. To mitigate this, methods either (a) only unroll a small number of steps (e.g. one epoch or a few updates) and rely on that to approximate the full effect ⁴³, or (b) derive analytic shortcuts. The use of Hessian-vector products is one such shortcut: rather than unrolling every step, one can sometimes capture the influence on final parameters via a matrix inversion or recursive formula that accounts for many steps compactly ³³ ³⁴. Still, these computations can be expensive for high-dimensional parameter spaces. Memory-efficient techniques like gradient checkpointing or reversing the training loop on the fly are required if long unrolls are attempted. Moreover, each target example

considered may require its own backward pass through the unrolled graph (though some methods, like Source or influence functions, can compute influences for all training points relative to one test point in a single solve using linear superposition) ³ ⁶. In summary, unrolling-based attribution can be orders of magnitude more costly than evaluating a trained model, which limits their practical use to smaller models or calls for approximation strategies.

Pros and cons (vs. classical/Bayesian influence): Unrolling approaches offer several clear advantages:

- **Capturing Training Dynamics:** They naturally account for *when* and *how* a data point is used during training. This means they handle cases of *underspecification* and data ordering effects that classical influence functions miss ¹² ¹⁶. For instance, if a critical sample is seen early vs. late in training, an unrolled method can assign different influence, whereas a classical method cannot distinguish those scenarios ¹⁷ ⁵². This is crucial for one-pass training on very large datasets (common in large language models) where order has a big impact ¹⁷.
- **No Need for Invertible Hessian:** Unrolling sidesteps the problematic requirement of inverting a possibly singular Hessian at optimum. It instead follows the actual gradient path, implicitly including some higher-order effects without needing an explicit Hessian inverse. This makes it more applicable to deep nets where $\$H\$$ might be ill-conditioned or huge ²³.
- **Multi-Stage and Non-Convergent Scenarios:** In practice, models are trained in stages (pre-training then fine-tuning, curriculum learning, etc.) or stopped early. Unrolling methods can be designed to handle these by incorporating the entire pipeline into the differentiation (or segmenting it) ¹² ²⁵. Classical influence, by contrast, struggles if the model hasn't truly converged or if the training objective changes partway.
- **Counterfactual Accuracy:** By hewing closer to the actual retraining outcome (in the limit of full unrolling), these methods can provide more accurate predictions of leave-one-out changes in loss ⁴¹. In scenarios where implicit methods like influence functions show large error, unrolling-based estimates tend to track the true effect better ¹² ⁴¹.

However, there are notable drawbacks:

- **High Computational Cost:** Differentiating through training is expensive. Even approximate unrolling (one epoch or truncated) is much slower than a single influence function computation. This limits unrolling methods' scalability to very deep or long-running training jobs. They might be infeasible for full attribution in models with billions of parameters or millions of steps, whereas Bayesian influence with MCMC (while also costly) might at least handle large models through sampling ²⁴.
- **Implementation Complexity:** Unlike influence functions which can reuse standard autograd operations for Hessian-vector products, unrolling often requires custom training loops or meta-learning frameworks to compute gradients of gradients. Implementing these without error can be tricky, and they may not be as readily available in off-the-shelf libraries. Maintaining numerical stability over a long unrolled sequence is also challenging (gradients can explode/vanish).

- **Approximation Error:** Truncated or approximate unrolling introduces its own error. If only the last epoch is unrolled, influences from earlier epochs are estimated coarsely ⁴³. There is a risk of misestimating influence if the truncation misses important interactions (though research like ACC-SGD-IE aims to minimize this bias ³⁵ ³³). In contrast, classical influence functions provide an exact analytic value *under their assumptions*. So when those assumptions hold (e.g. convex logistic regression), classical influence can be very precise ¹⁴, whereas unrolling might be overkill.
- **Sensitivity to Training Noise:** While unrolling captures one particular trajectory of training, it inherits that trajectory’s idiosyncrasies. Random seed changes or different SGD noise could yield different influence outcomes, meaning one might need to average over multiple unrollings (which is rarely done due to cost). A Bayesian approach, by treating influence as a distribution ²⁰, is more explicit about this variability. Unrolling methods usually don’t account for run-to-run variance unless one explicitly marginalizes over initializations.

Applicability: Optimizer-unrolling attribution is *in principle model-agnostic* – it can be applied to any model trained with gradient-based optimization, including deep neural networks and transformers. In practice, however, its applicability is constrained by resource demands. For smaller models (e.g. a moderate CNN on CIFAR-10 or a small transformer on a classification task), one can feasibly unroll a few epochs of training for influence analysis ⁵³. For very large models (e.g. GPT-scale transformers), fully unrolling even one epoch is impossible, but one might apply these techniques to a *fine-tuning stage* or a specific subset of parameters. Recent work tries to push the scalability: for example, the local BIF method showed it’s possible to handle billions of parameters by avoiding explicit unrolling and using clever Bayesian approximations ²⁴. In scenarios like continual learning or multi-stage training, unrolling methods shine because they can attribute influence to specific stages. For instance, one could quantify how a data point in pre-training versus in fine-tuning each contributed to a downstream prediction by unrolling those stages separately and comparing. Classical influence functions would have trouble partitioning influence by stage, since they usually treat the final model monolithically. Overall, unrolling methods are most applicable in research settings or targeted diagnostics where high fidelity influence tracing is needed on models that are small enough (or training runs short enough) to permit it.

Gradient Trajectory-Based Attribution Techniques (TracIn and Beyond)

Gradient trajectory-based methods take a different approach: instead of explicitly differentiating through the training procedure, they **leverage information from the model’s gradient trajectory** to estimate influences. The prototypical example is **TracIn** (Pruthi et al., 2020), which stands for “*Tracing Influence*” by following gradient descent ⁵⁴. The core idea of TracIn is to accumulate the influence of a training example by summing up how much that example’s gradient aligned with the gradient of a target test point at various points in training ⁵⁴ ⁵⁵. Intuitively, if a particular training sample z_i tends to *reduce* the loss of a test example z_{test} when z_i is learned (e.g. their gradients point in similar directions in parameter space), then z_i has a strong positive influence on z_{test} ’s prediction ⁵⁵. Conversely, if training on z_i often increases the test point’s loss (gradients point in opposite directions), it has a negative influence. TracIn formalizes this by summing the dot-products of gradients across training checkpoints:

$$\text{TracIn}(z_i, z_{\text{test}}) = \sum_{t \in \mathcal{T}} \eta_t \langle \nabla_{\theta} L(z_{\text{test}}; \theta_t), \nabla_{\theta} L(z_i; \theta_t) \rangle,$$

where θ_t are the model parameters at training step or checkpoint t , and η_t is the learning rate at that step ⁵⁶. In practice, one uses a set of stored checkpoints $\mathcal{T}$ (e.g. periodic snapshots of the model) rather than every iteration. Each term $\langle \nabla L_{\text{test}}, \nabla L_i \rangle$ indicates how much the training example’s gradient would affect the test loss if applied at that point – a positive inner product means that step on z_i would decrease test loss, contributing a positive influence ⁵⁵. By summing these contributions, TracIn approximates the total influence of including z_i in training, via first-order effects along the trajectory. Notably, TracIn does **not** require any Hessian computation or second-order derivatives at all; it uses only the same first-order gradients that the optimizer uses ⁵⁵. This simplicity was a major selling point: “*all it needs is the ability to work with gradients, checkpoints, and loss functions*”, making it easy to implement and broadly applicable ⁵⁷. In fact, TracIn is architecture-agnostic – it can be used with any model trained by (stochastic) gradient descent, including deep networks, since it only relies on gradient information and saved model states ⁵⁸.

Key methods and developments: Following TracIn, there have been other trajectory-based attribution techniques and improvements. One variation is *TracInCP* (“TracIn with Checkpoints”), which specifically chooses a fixed set of checkpoints (say, after each epoch or at several evenly spaced iterations) to evaluate the gradients. This reduces computational load while still sampling the trajectory. There is also the idea of **cherry-picking layers** ⁵⁹ – computing the gradient inner product on only certain layers (e.g. the last layer of the network) instead of the full parameter vector, to save time and mitigate high-dimensional noise. Empirically, focusing on final-layer gradients can still highlight influential training examples (since final-layer gradients relate closely to predictions) while simplifying the computation ⁵⁹.

Beyond TracIn, researchers have sought to capture *temporal aspects* of data influence more explicitly. Wang et al. (2024) formalized the notion of **trajectory-specific leave-one-out (LOO) influence**, which asks how removing a data point *at the specific time it occurs in a given training run* would alter the final model ⁶⁰ ¹. This concept acknowledges that removing a sample earlier vs. later leads to different outcomes, and it directly violates the permutation-invariance assumption of classical influence functions ⁶¹ ⁴. To approximate this trajectory-dependent influence efficiently, they introduced **Data Value Embedding (DVEmb)** ⁶² ⁶³. The idea is to compute a *compact embedding vector* for each training point that accumulates its interactions with the model across the entire training process ⁶⁴ ⁶³. Remarkably, once you have this embedding (essentially a summary of how that point’s gradient influenced the model over time), the influence on any test query can be obtained by a simple dot product between the training point’s embedding and the test point’s gradient at the end ⁶⁵ ⁶³. This allows **real-time influence queries** after training – one can attribute model decisions to training data on the fly, without retraining or heavy computation, by referencing these stored embeddings ⁶⁶ ⁶³. To scale this to deep models, they propose *influence checkpointing*, which computes partial embeddings at multiple checkpoints in parallel and combines them ⁶⁷. This not only speeds up embedding computation but also enables tracking how a data point’s influence *changes over the course of training* ⁶⁸. Using this approach, they uncovered that training typically has distinct phases of influence: an initial burst where influence is high, a long middle period of low influence (a “basin”), and a late phase where influence rises again as the model fine-tunes ⁶⁹. Such insights are hard to get from classical or static methods.

Another trajectory-based method is **Diff-In** (introduced earlier), which can also be viewed in this category. Diff-In explicitly accumulates influence *differences* between steps ⁴⁴. It essentially tracks the trajectory of influence by computing how each step’s gradient update changes the influence score, summing these changes up. Because it leverages second-order approximations per step, Diff-In manages to remain accurate without assuming convexity ⁴⁵. We also see hybrid approaches: for example, some works

consider *last-iter influence* – only tracking the influence contributions in the final epoch, as an efficient proxy for total influence ⁴³. While not as principled as full-trajectory summation, focusing on later training stages can capture which examples ultimately had the largest impact on the converged model.

Computational requirements: Gradient trajectory approaches are generally more lightweight than full optimizer unrolling, but they have their own costs. The main expense is **gradient computation at many checkpoints**. If we use M checkpoints and want to compute influence of all N training points on a particular test point, a straightforward implementation would require computing N gradients at each of M checkpoints, for MN gradient evaluations. TracIn often mitigates this by restricting to a subset of layers or by computing dot-products in a single backward pass if possible (for example, one can vectorize operations for a batch of training points at once at a given checkpoint). Storing gradients can also be heavy: a single model checkpoint might involve millions of parameters, so storing full gradient vectors for many data points and many checkpoints is memory-intensive. This is why methods like TRAK (Ilyas et al., 2022) use random projections to compress gradients into a lower-dimensional *signature* without too much loss of information ⁷⁰ ⁷¹. TRAK essentially projects each high-dimensional gradient into a smaller space and then uses those projections to compute influence scores, improving scalability. Data value embeddings likewise condense a sequence of gradients into one vector per data point ⁶³.

In practice, one often balances **granularity vs. cost**. Using every training iteration would be most precise but exorbitant for long runs. Instead, using say one checkpoint per epoch, or a handful of strategically chosen points (e.g. beginning, middle, end of training), provides a reasonable approximation. Pruthi et al. demonstrated that even with modest numbers of checkpoints, TracIn could identify influential examples that matched those found by brute-force leave-one-out retraining ⁵⁴ ⁵⁵. As another optimization, one can focus on *influential candidates* rather than all points: for example, first filter potential outliers or high-loss examples, then apply trajectory methods to those to see if they truly have high influence on a validation set. The embedding approach by Wang et al. suggests an efficient route – by the time training is done, you have an embedding, and influence queries become trivial dot-products ⁶³. Computing the embeddings still requires integrating gradients over time, but with checkpointing and parallelization this was reported to be *faster than the most efficient influence function implementation* for large models ⁷².

Pros and cons (vs. classical/Bayesian influence): Gradient-based trajectory methods offer a compelling middle ground between simplicity and accounting for training dynamics:

- **Pros:**

- *No Hessian Inversion:* These methods avoid computing or inverting Hessians altogether ²³. This eliminates a major failure point of classical influence in deep nets (non-invertible or huge Hessians). All computations are first-order (gradients), which are readily obtained from autograd and scale linearly with model size.
- *Handles Non-Convex and Order Effects:* Like unrolling methods, gradient trajectory approaches work on the actual training process, so they naturally handle non-convex loss surfaces and do not assume permutation invariance of data ¹⁶ ¹⁷. They recognize that seeing data point A before B may lead to a different outcome than vice versa, and the method's output can reflect that because it uses the actual sequence.
- *Relative Efficiency and Simplicity:* Compared to full unrolling differentiation, methods like TracIn are relatively easy to implement and computationally cheaper. One can often piggyback on training: e.g. save occasional checkpoints and reuse them to compute gradients post-hoc. The requirements

(compute gradient of loss on single examples) are standard operations in deep learning frameworks, making these methods accessible to practitioners ⁵⁷ .

- *Scalability Strategies*: There are built-in ways to scale trajectory methods – subsample checkpoints, project gradients, focus on top layers, etc. These allow the method to be used in larger-scale settings than a full unroll. For instance, Ilyas et al. showed data attribution on ImageNet-scale models using TRAK by compressing gradients with random projections ⁷⁰ ⁷³ . Wang et al. argued their embedding method can handle foundation model training, offering faster runtime than even optimized influence function code ⁷² .
- *General Applicability*: As noted, gradient tracing can be applied to any model trained with SGD or its variants ⁵⁸ . It's not tailored to a specific architecture. It also works for *batches* of data – one can sum up gradient dot-products for all examples in a batch to estimate that batch's overall influence on a test point, which is useful if we suspect certain mini-batches (perhaps those containing outliers) had outsized effects.

- **Cons:**

- *Approximation Accuracy*: Trajectory methods provide a *heuristic approximation* to influence. They assume that the dot product of gradients is a good proxy for how a small parameter step on one example affects another's loss. If learning rates are large or interactions are highly non-linear, this approximation may be off. They also usually neglect second-order effects (except methods like Diff-In which try to include some). Thus, they might not perfectly rank influences in all cases. For example, if two training points never directly appear in the same neighborhood of training (due to shuffling), their interaction might be missed by coarse checkpoint sampling.
- *Storage and Compute Overhead*: While easier than unrolling, storing model checkpoints (which can be gigabytes each for large nets) and computing many gradients can still be burdensome. If one wants per-sample influences for *all* training data on *all* test points of interest, the computation can blow up (though typically one narrows the focus to a specific test example or a handful of them for analysis).
- *Noise and Variance*: Because these methods rely on gradients from the actual training run, they can be sensitive to noise in those gradients. SGD randomness or even minor changes in hyperparameters can cause gradient dot products to fluctuate. This means the influence scores might not be stable across training runs (similar to the concern raised in the Bayesian perspective ²¹). Smoothing techniques, or aggregating over multiple runs, could improve reliability but add cost. Classical influence functions, by looking at an optimal point, sometimes bypass short-term noise (but have their own issues if the optimum isn't reached).
- *Limited Theoretical Guarantees*: Unlike classical influence which has roots in Cook's theory with known error bounds under assumptions, the theoretical understanding of methods like TracIn is still developing. Recent work has started analyzing the error of unrolling and first-order trajectory approximations ⁷⁴ ⁷⁵ , but generally one uses these methods because they *work empirically*, not because we have a strict bound on their accuracy in deep networks. This may give some practitioners pause, though ongoing research is strengthening their theoretical foundations.

Applicability: Gradient trajectory methods are **broadly applicable** across model types and training regimes. They have been demonstrated on vision CNNs, NLP models, and even non-deep models, as long as one can compute gradients. For transformers and other very large models, the challenge is mainly practical: computing and handling enormous gradients. Yet, the random-projection trick used in TRAK ⁷¹ and the efficient embedding in DVEmb ⁶³ show that even such cases can be tackled with clever engineering. A salient point is that trajectory methods can be applied *retrospectively*. This means one can take an already-

trained model (along with some saved states during its training) and analyze influences without needing to modify the training procedure itself. This is attractive for real-world workflows, as one can add influence analysis after the fact. For example, a company training a large transformer on millions of documents might periodically save model checkpoints and gradient information for a small subset of examples. Later, if an issue arises with the model's predictions, they could use those saved states to perform a TracIn-style analysis to find which training data were most responsible, all without having to re-train the model or instrument the entire training loop with second-order gradients. This “plug-and-play” nature makes gradient trajectory methods a practical choice for integrating into existing pipelines. In contrast, unrolling-based methods often require designing the training process with differentiation in mind (or at least re-running some version of training), which might not be feasible for a once-off large training job.

Comparative Analysis of Unrolling vs. Trajectory Methods

Both optimizer-unrolling and gradient trajectory-based approaches aim to illuminate how training data shape the final model, but they do so in distinct ways with different strengths. Here we compare the two families along key dimensions:

- **Theoretical Foundation:** Unrolling methods derive from an exact *implicit function* view of training – they attempt to compute or approximate the true change in final parameters or loss when a training point is perturbed. In principle, a full unrolling gives the *ground-truth* influence (subject to numerical error) for even complex training setups. Trajectory methods, conversely, rely on a *first-order approximation* along the training path. They don't simulate removal of a point, but rather measure correlation of gradients as a surrogate for influence. Thus, unrolling is more rooted in explicit causation (what *would* happen if removed), while trajectory methods infer influence from observed gradient dynamics (what *did* happen during training). In practice, both are approximations (since full unrolling is rarely done), and both correlate well with true leave-one-out effects under many conditions ⁴¹.
- **Accuracy vs. Efficiency:** Optimizer-unrolling tends to be more accurate (especially for tricky scenarios) but much less efficient. For example, if a model is not fully trained to convergence, classical influence might be invalid, but an unrolled method like Source can still estimate influence faithfully ¹² ⁴¹. Trajectory methods sacrifice some accuracy (they might miss higher-order interactions) for the sake of scalability – using only gradients is cheaper than computing Hessians or unrolling thousands of updates. Empirical studies often find that simple methods like TracIn closely match the results of more expensive influence calculations in identifying top influential examples, albeit with some noise ⁵⁴. If absolute precision is needed (say for legal requirements like data deletion audits), one might lean toward unrolling or at least a second-order method; if one needs a quick heuristic to flag suspicious data, trajectory methods suffice.
- **Handling of Training Order and Dynamics:** Both families outperform classical static methods in accounting for training dynamics, but they do so differently. Unrolling inherently includes order because you differentiate through the specific sequence of updates – if the sequence changes, the gradients change accordingly. Trajectory methods explicitly use the sequence of model states and hence also capture ordering effects ¹⁷ ⁵². Notably, trajectory methods can highlight *when* during training a data point was most influential (especially approaches like DVEmb that track influence over time) ⁶⁹. Unrolling methods typically output a single influence score per point for the entire training run, although one could conceivably unroll segment by segment to see stage-wise influence.

So for understanding *stagewise* or temporal influence patterns, trajectory methods provide a more direct toolkit.

- **Complexity in Implementation:** Trajectory approaches are generally easier to deploy. They require access to model checkpoints and the ability to compute gradients on those – tasks well-supported by ML frameworks. Optimizer unrolling might require custom backward passes through the training algorithm, which can be complex, especially if the training involves non-differentiable operations or data augmentations (one might have to approximate those or treat them as fixed). Unrolling across multiple epochs with shuffling also needs care: one approach is to treat the random seed as fixed so that the same sequence is differentiable; this can complicate typical training pipelines.
- **Batch-Level Influence:** In some real workflows, we care about identifying not just single harmful examples, but whether certain *batches* or data sources are problematic. Both methods can be extended: unrolling can differentiate through a whole batch's inclusion/exclusion (one can assign a weight to a batch and get the gradient of final loss w.r.t that weight). Trajectory methods can sum gradient dot-products for all points in a batch to measure that batch's influence on a test point or validation loss. If we suspect an entire mini-batch (maybe from a specific data shard) caused a training instability, trajectory methods let us inspect its effect at the time it was used. Unrolling could simulate leaving out that batch entirely, though that borders on doing a partial retraining.
- **Bayesian vs. Frequentist Usage:** Unrolling and trajectory methods as described are typically used in a single-run (frequentist) context: given one training run, compute influence. A Bayesian mindset would consider averaging these influence estimates over many runs or treating them as random. In principle, one could combine approaches: e.g. run TracIn on multiple independent training runs and see which training examples consistently rank highly influential (as suggested by Nguyen et al. ²¹). This would yield more robust conclusions, albeit at a high computational cost. Bayesian influence function methods (like BIF ⁷⁶) take a different route by incorporating uncertainty via sampling rather than multiple full training runs, which is more efficient than brute-force repetition but still expensive. In comparison, trajectory methods are lightweight enough that repeating a few runs might be feasible for smaller models to gauge influence variance.

To sum up, **optimizer-unrolling and gradient-trajectory methods are complementary**. Optimizer-unrolling gives a more *simulation-based* view: “what is the causal effect on the final model if I change this training example or its weight?” Gradient trajectory methods give a more *diagnostic* view: “during training, how did this example's gradient interact with others and the evolving model?” In scenarios where trust and exactness are paramount (e.g. deciding whether to remove a data point due to its harmful influence), one might use unrolling-based techniques or verify trajectory-based insights with a follow-up retraining. In exploratory analysis or on-the-fly debugging, trajectory methods often provide a quick answer.

Applications in Data Quality, Debugging, and Interpretability

Understanding data influence is not just a theoretical exercise – it has important real-world applications in machine learning practice:

- **Data Quality Assessment and Cleaning:** One of the primary uses of influence methods is to identify *mis-labeled or low-quality training data*. Points that strongly negatively impact validation performance are likely mislabeled or noisy. For example, Koh & Liang (2017) showed that removing a

small set of training points with high harmful influence (as identified by influence scores) improved test accuracy ⁷⁷. Influence functions and their modern counterparts have been used to flag outliers or label errors for review. Hara et al. (2019) demonstrated that their SGD-influence method could pinpoint such problematic instances and that removing them yielded a better model ⁷⁸ ⁵³. In large datasets, manually inspecting every sample is infeasible; influence scores let practitioners rank and prioritize candidates for cleanup. In production ML, one might periodically run influence analysis to catch data drift or newly introduced bad data – e.g. if a new data source starts contributing highly influential outliers that hurt model performance, it can be detected and filtered.

- **Model Debugging and Failure Analysis:** When a model makes an unexpected or incorrect prediction, influence methods can help answer *“why did the model predict this?”* By identifying the training examples that most influenced that specific prediction, we can often find enlightening patterns. For instance, suppose a vision model mistakenly classifies an image; a TracIn analysis might reveal that a particular training image with similar but wrongly labeled content had a strong positive influence on the mistake. This could indicate a training data issue (the model learned the error from a mislabeled sample) or a bias (the model latched onto a spurious feature present in both the training and test image). Such insights are valuable for debugging model behavior. In NLP models, influence tracing can uncover if a model answer was primarily driven by memorizing a specific document in the training set – a key consideration for detecting data leakage or privacy violations. Indeed, one of the usage examples in recent literature is **data leakage detection** ⁷⁹: by tracing predictions back to training data, one can check if a supposedly confidential test entry was inadvertently seen during training (if the test has a near-duplicate in training with huge influence, that’s a red flag ⁸⁰ ⁷⁹). Additionally, *prediction brittleness* or model instability can be studied by seeing which training points contribute to high-variance outputs ⁷⁹. If a model’s output changes drastically with minor input changes, influence methods might reveal a small set of training samples that are over-dominating the decision boundary.
- **Interpretability and Transparency:** In domains like healthcare or legal AI, being able to *explain a prediction* is crucial. Training data attribution offers a form of example-based explanation: *“This model prediction was largely influenced by these prior cases in the training data.”* This can sometimes be more interpretable to humans than feature importance scores. For example, a medical diagnosis model might explain a classification by pointing to a few past patient examples (from training) that had similar symptoms and outcomes, which heavily influenced its decision. This helps build trust, as doctors can review those cases for plausibility. In generative models, data attribution can identify which training data regions contribute to a certain generated content, which is relevant for copyright and fairness auditing ⁸¹. If a generative model produces a certain stylized output, influence analysis might trace that to a specific artist’s work in the training set, raising intellectual property considerations ⁸¹. In data marketplaces, where datasets are bought and sold, influence scores could form the basis of **data valuation** – quantifying how much each contributor’s data boosted (or harmed) the model’s performance ⁸¹. Methods like Shapley value-based data valuation are conceptually linked to influence, but influence methods provide more efficient proxies for those in complex models.
- **Coreset Selection and Active Learning:** By understanding influence, one can try to select a subset of training data that is most informative. If many training points have near-zero influence on the validation set, they might be redundant and could be pruned to create a smaller “coreset” without much performance loss. Some researchers have used influence scores to guide data subset selection

for efficient training ⁸² ⁸³. Similarly in active learning, one could prioritize labeling new data points that are expected to have high influence on the current model's predictions (though this is a somewhat inverted use of the concept).

- **Monitoring and Improving Training Dynamics:** Influence methods can be used during training to monitor which data are driving learning at different stages. For example, an influence analysis might show that early in training, a certain cluster of examples (maybe easy ones) are guiding the initial parameter changes, whereas later, another set (hard examples or edge cases) become more influential. This can inform curriculum learning – e.g. ensure that important examples appear at the right time. It can also identify if the model becomes too dependent on a small subset of data (which could signal overfitting). With techniques like data value embedding, one could imagine real-time dashboards that flag when a single batch or example is dramatically steering the model (which might indicate a problematic gradient or a need to adjust learning rate).

Looking at **real-world ML workflows**, integrating these methods requires some effort but is increasingly feasible. For smaller scale (research or modest industrial models), open-source implementations of influence functions and TracIn are available, and one can run them after training to produce reports on influential training samples ⁸⁴ ⁸⁵. For larger scale, companies may incorporate influence logging into their training pipeline: e.g., every N steps, compute and record the gradients of a small reference set of validation examples with respect to current model parameters. These gradients can later be combined with per-batch training gradients (if stored or reproducible) to compute influence approximations. Some organizations have also begun using influence analysis for model audit trails – recording which training data most affected a particular decision, which can be important for accountability. As ML models continue to grow in size and are deployed in critical areas, the ability to *attribute outcomes back to training data* will be key for diagnosing failures and ensuring compliance (e.g., with “right to be forgotten” requests, one might need to gauge the impact of removing a user's data without fully retraining a 100-billion parameter model – influence estimates could guide efficient *unlearning* procedures).

Conclusion

Training data attribution during training is a rich and evolving field at the intersection of optimization and interpretability. **Optimizer-unrolling methods** provide a principled (if costly) way to differentiate through the learning process and capture subtle, trajectory-dependent influences that classical post-hoc methods miss. **Gradient trajectory methods** like TracIn offer a practical, first-order means to trace influence using information readily available from training dynamics. Both approaches improve upon classical influence functions by accounting for the realities of modern training – non-convexity, early stopping, and data ordering – albeit via different strategies. Optimizer-unrolling techniques shine in accuracy and theoretical fidelity, while gradient-based techniques excel in simplicity and scalability. The choice between them can be guided by the problem at hand: for deep dive analyses on smaller models or critical examples, unrolling-based attribution (or hybrid methods like Source and Diff-In) may be warranted; for broad analysis of large-scale models, trajectory-based methods (possibly augmented with efficient embeddings or projections) are more viable.

In many cases, these methods are complementary. A practitioner might use cheap gradient-based scores to filter potentially influential or suspicious data, then apply a more exact unrolling method to those top candidates for confirmation. As research continues, we see convergence of ideas – for example, unrolling-based formulas being computed with influence-function-like efficiency ³⁸, and gradient methods

incorporating higher-order corrections ⁴⁵. The ultimate goal is to make understanding data influence **accurate, scalable, and routine**. When achieved, this will greatly enhance our ability to debug models, curate datasets, and ensure that our models learn from data in the ways we expect. Recent literature shows we are making significant strides toward that goal, developing tools that let us peer into the training process and answer the vital question: *“Which training data made my model do that?”* – with quantifiable evidence to back it up.

Sources:

- Koh, Pang Wei, and Percy Liang. *“Understanding Black-box Predictions via Influence Functions.”* ICML 2017.
- Hara, Satoshi, and Kohei Hayashi. *“Data Cleansing for Models Trained with SGD.”* NeurIPS 2019 ²⁹ ³⁰.
- Pruthi, Garima, et al. *“Estimating Training Data Influence by Tracing Gradient Descent.”* NeurIPS 2020 ⁵⁴ ⁵⁵.
- Ilyas, Andrew, et al. *“Datamodels: Predicting Predictions from Training Data.”* ICML 2022 ⁸⁶ ⁸⁷.
- Ilyas, Andrew, et al. *“TRAK: Attributing Model Behavior at Scale.”* NeurIPS 2022 ⁷⁰ ⁷¹.
- Bae, Juhan, et al. *“Training Data Attribution via Approximate Unrolled Differentiation.”* NeurIPS 2024 ¹² ³⁸.
- Tan, Haoru, et al. *“Diff-In: Data Influence Estimation with Differential Approximation.”* ICLR 2025 ⁴⁴ ⁴⁶.
- Wang, Jiachen, et al. *“Capturing the Temporal Dependence of Training Data Influence.”* arXiv 2024 ⁶⁰ ⁶³.
- Kreer, Philipp, et al. *“Bayesian Influence Functions for Hessian-Free Data Attribution.”* arXiv 2025 ⁷⁶.
- Nguyen, Elisa, et al. *“A Bayesian Approach to Analyzing Training Data Attribution in Deep Learning.”* NeurIPS 2023 ⁸⁸.
- Shi, Yunxiao, et al. *“Accumulative SGD Influence Estimation for Data Attribution.”* arXiv 2025 ³³ ³⁴.
- ICLR Blogpost. *“Unraveling the Impact of Training Samples.”* 2024 ⁷⁹ ⁹.

¹ ⁴ ⁵ ¹⁶ ¹⁷ ¹⁸ ⁵² ⁶⁰ ⁶¹ ⁶² ⁶³ ⁶⁴ ⁶⁵ ⁶⁶ ⁶⁷ ⁶⁸ ⁶⁹ ⁷² ⁷⁴ ⁸¹ Capturing the Temporal Dependence of Training Data Influence

<https://arxiv.org/html/2412.09538v1>

² ³ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹⁴ ¹⁵ ⁷⁰ ⁷¹ ⁷³ ⁷⁹ ⁸⁰ ⁸⁶ ⁸⁷ Unraveling The Impact of Training Samples | ICLR Blogposts 2024

<https://iclr-blogposts.github.io/2024/blog/unraveling-the-impact-of-training-samples/>

¹² ²⁵ ³⁸ ³⁹ ⁴⁰ ⁴¹ [2405.12186] Training Data Attribution via Approximate Unrolled Differentiation <https://arxiv.org/abs/2405.12186>

¹³ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ⁴² ⁵³ ⁷⁷ ⁷⁸ papers.neurips.cc
<http://papers.neurips.cc/paper/8674-data-cleansing-for-models-trained-with-sgd.pdf>

¹⁹ ²⁰ ²¹ ²² ⁸⁸ NeurIPS Poster A Bayesian Approach To Analysing Training Data Attribution In Deep Learning
<https://neurips.cc/virtual/2023/poster/71357>

²³ ²⁴ ⁷⁶ [2509.26544] Bayesian Influence Functions for Hessian-Free Data Attribution
<https://arxiv.org/abs/2509.26544>

31 32 33 34 35 Accumulative SGD Influence Estimation for Data Attribution

<https://arxiv.org/html/2510.26185v1>

36 37 NeurIPS 2024 Paper Alert ➡ Paper Title: Training Data Attribution ...

https://www.linkedin.com/posts/naveen-manwani-65491678_neurips-2024-paper-alert-paper-title-activity-7271471090516799489-JTDS

43 [PDF] Training Data Attribution via Approximate Unrolling - NIPS papers

https://proceedings.neurips.cc/paper_files/paper/2024/file/7af60ccb99c7a434a0d9d9c1fb00ca94-Paper-Conference.pdf

44 45 46 47 48 49 82 83 Diff-In: Data Influence Estimation with Differential Approximation | OpenReview

<https://openreview.net/forum?id=Jds4tiTo2a>

50 51 [PDF] Understanding Training Data in Large-Scale Machine Learning

https://www.lti.cs.cmu.edu/research/dissertations/sangkeuc_phd_lti_2024.pdf

54 55 56 57 58 59 [2002.08484] Estimating Training Data Influence by Tracing Gradient Descent

<https://arxiv.org/abs/2002.08484>

75 [PDF] arXiv:2412.09538v1 [cs.LG] 12 Dec 2024

<https://arxiv.org/pdf/2412.09538>

84 sato9hara/sgd-influence: Python codes for influential ... - GitHub

<https://github.com/sato9hara/sgd-influence>

85 How Data Influence Changes in - Pangram Labs

<https://www.pangram.com/history/60477b82-33e4-41e3-bc1f-7c866509ddb2>